Research Intelligence Engine: Architecture and Empirical Characterization of a Desktop Multi-Source Scholarly Retrieval and LLM-Annotation Pipeline

Suryavansh Jaggi
IILM University, Greater Noida
suryavansh.jaggi.cs27@iilm.edu

***Abstract*—Open scholarly-metadata services make it practical to build local literature tools that query many providers at once, yet little is known about how such clients behave when providers are slow, fail, or disagree about metadata. We characterize the Research Intelligence Engine (RIE), an open-source Electron/PostgreSQL desktop application that fans a query out to six providers, normalizes and deduplicates the results, stores them, and sends abstracts to a hosted LLM. Evidence is kept in separate classes. Controlled experiments run RIE's unmodified modules against a deterministic emulator of the upstream APIs (the live APIs were unreachable from our cloud sandbox) and a real PostgreSQL server. Search latency tracks the slowest adapter within 11 ms, but provider throttles impose a 350 ms floor and up to ≈2.6 s of delay on back-to-back searches, a hung provider blocks all results until its 15–30 s timeout, and arXiv failures are reported as empty successes. The deduplicator is order dependent (1 000 or 2 000 output records for identical inputs) and merges DOI-less records only when metadata strings match up to letter case. The AI client is serial, spaced by 2 s, retries only HTTP 429, and sends the literal text "null" for missing abstracts. A separate, single, user-recorded live session on a Windows desktop (query "transformers": four providers returned two papers each, Semantic Scholar returned HTTP 429, and eight papers were analyzed and saved in 27 s) is consistent with these findings but is not a benchmark. Contributions are engineering and empirical; no retrieval-quality or LLM-quality claims are made.**

***Index Terms*—scholarly metadata aggregation, federated search, record deduplication, LLM-assisted literature analysis, Electron, fault injection**

I. Introduction

Researchers who need to survey a topic increasingly query open scholarly-metadata services directly rather than a single curated index: OpenAlex [1], Semantic Scholar [2], [3], Crossref [4], Europe PMC [5], PubMed through the NCBI E-utilities [6], and arXiv [7]. Coverage, metadata completeness and citation counts differ between such sources [8]–[10], so a tool that queries several of them must (i) reconcile heterogeneous schemas, (ii) tolerate partial failure and provider rate limits, and (iii) remove duplicates that arise because the same work is indexed by several providers under different identifiers. Separately, large language models (LLMs) are now used to summarize individual papers, with a well-documented risk of unsupported output [11], [12].

The Research Intelligence Engine (RIE) is an open-source desktop application (Electron + PostgreSQL) that packages these functions into one workflow: concurrent retrieval from six providers, normalization into a canonical record, in-memory deduplication, relational persistence with PDF acquisition, abstract-level LLM analysis through a hosted inference API, and a Markdown/LaTeX editor. To our knowledge, version 1.0.0 of the code base has no published evaluation, and its README describes a narrower and partly different system than the code implements (Section V).

This paper reports what can be established about RIE from its source code and from controlled experiments on its actual modules. We pose four research questions.

- **RQ1** How does the implemented fan-out strategy determine end-to-end search latency and failure containment?
- **RQ2** How reliably does the three-key deduplication resolve cross-provider duplicates, and which metadata disagreements defeat it?
- **RQ3** What do the local components (normalization, persistence, library indexing) cost, and how do they scale?
- **RQ4** How does the AI-analysis client schedule requests, recover from errors, and construct its prompt?

Scope and evidence classes. The cloud sandbox in which the experiments were run blocks outbound connections to the scholarly APIs and to the LLM endpoint. The repository's real source adapters, normalizers, deduplicator, orchestrator, database layer, and AI client were therefore executed unmodified against a deterministic *emulator* of the upstream HTTP interfaces (with injected latency and faults) and a real local PostgreSQL 16 server; these results are labelled **[S]** (simulated upstreams) and are reproducible with the released scripts. In addition, the author recorded one *live* session of the Windows desktop application from its in-app logs; these observations are labelled **[L]**, were not independently reproduced, and are reported separately and never mixed with [S] results (Section IV, Table II). We did *not* measure real provider latency distributions, retrieval effectiveness (precision/recall/nDCG), real cross-provider overlap rates, or LLM output quality, and no such number appears in this paper.

Contributions. (1) A code-grounded description of RIE's architecture, data model and pipelines, including an audit of what is implemented, validated and merely documented. (2) A reproducible harness (fetch emulator, synthetic ground-truth corpus, fault injection, single-factor metadata perturbation) that runs the unmodified modules, together with a claim-by-claim audit of every reported number against the raw results and an independent re-run. (3) Empirical findings on latency structure, failure reporting, request amplification, and deduplication behaviour, notably that the repository's deduplicator is order-dependent (non-confluent) and that its fallback key fails under every single-factor metadata disagreement we tested except letter case. (4) A prototype relaxed-key resolver, implemented only inside the harness, whose precision/recall trade-off is quantified. (5) A clearly labelled single live session of the real application, compared qualitatively with the emulated findings. *Novelty statement:* the contributions are predominantly engineering and empirical; we do not propose a new algorithm, and union-find resolution and fielded matching are standard in record linkage [13]–[15].

The remainder of the paper is organized as follows: Section II reviews related work, Section III describes the system, Section IV formalizes the pipeline and the experimental design, Section V audits the implementation, Section VI reports experiments, and Sections VII to IX discuss, bound and conclude.

II. Related Work

Open scholarly metadata. OpenAlex was introduced as an open replacement for Microsoft Academic Graph [1]; the Semantic Scholar corpus and data platform expose papers, citation edges and full-text-derived fields [2], [3]; Crossref aggregates publisher-deposited DOI metadata [4]; Europe PMC and PubMed cover the life sciences [5], [6]. Comparative studies show non-trivial differences in coverage and reference completeness among bibliographic sources [8], [10] and in their suitability for systematic searching [9]. Only part of the literature is openly accessible (an estimated 28 % or more in a 2018 study [16]), which motivates RIE's per-provider PDF-link handling. These studies evaluate the sources themselves rather than a local client that merges them.

Federated search and tail latency. Querying several independent engines and merging the results is the classical federated- (meta-) search problem, whose main sub-problems are resource description, resource selection and result merging [17]; rank-fusion methods such as reciprocal rank fusion combine per-source rankings [18]. RIE performs no score fusion: it keeps the top-k of each provider and re-sorts the union by year or citation count (Section III). A synchronous fan-out that waits for every backend is exposed to straggler effects: occasional slow components can dominate the overall latency of a fan-out service, which is the central concern of "tail-tolerant" design [19]. We examine that exposure for RIE concretely in Section VI.

Duplicate detection. Record linkage was formalized probabilistically by Fellegi and Sunter [13]; surveys and textbooks catalogue field-similarity metrics, indexing (blocking) and classification techniques for duplicate detection [14], [15]. Bibliographic de-duplication has been studied in the systematic-review setting: a multi-stage, fielded procedure for a reference manager has been described [20], and a purpose-built tool was reported to detect 42.9 % more duplicates than that manager's automatic de-duplication on a 1 988-citation benchmark [21]. RIE uses exact-equality keys (DOI, `source:source_id`, and a hash of title, first author and year). Equality keys are fast and precise but brittle under metadata variation; the experiments in Section VI quantify this brittleness for RIE specifically.

LLM-assisted literature analysis. Retrieval-augmented generation grounds a generator in retrieved passages [22], [23]; PaperQA and PaperQA2 apply this to full-text scientific articles with evidence gathering and cited answers [24], [25]; scite classifies the intent of citation statements [26]. Unsupported generation ("hallucination") is a documented failure mode of language generation [11], including fabricated references in literature-review writing [12]. RIE's analysis path is far simpler than these systems: it sends only the title and abstract plus a free-text instruction to a hosted open-weight model (default `openai/gpt-oss-120b` [27]) and performs no retrieval, grounding or verification. We evaluate only its request scheduling and construction, not output quality.

Positioning. We do not benchmark RIE against other tools, and we make no comparative performance claim. The paper's purpose is to characterize one system rigorously, with the harness and negative findings made available for reuse.

III. System Architecture

A. Process model

RIE follows Electron's multi-process design (Figure 1). Renderer windows (the main workbench and a separate editor window) run with `contextIsolation` enabled and `nodeIntegration` disabled; they reach the main process only through the `window.electronAPI` object that `preload.js` exposes via `contextBridge`, with namespaces `library`, `papers`, `ai`, `document`, `window` and `settings`. The main process (`electron-main.js`, plus `main/ipc/document-ipc.js`) registers one `ipcMain.handle` channel per operation and delegates to service modules. `paper-service.js` is the sole intermediary to ingestion, the database and the filesystem; `ai-service.js` wraps the hosted LLM endpoint; `document-service.js` spawns Pandoc for export. User-supplied API keys are persisted in a plaintext `settings.json` in the Electron `userData` directory and mirrored into `process.env`.

B. Source adapters and the canonical record

Each provider has an *adapter* (HTTP request and parsing only) and a *normalizer* (mapping to a canonical record, never throwing, missing fields → `null`). The registry in `search-orchestrator.js` binds names to pairs of functions, so adding a provider requires one entry plus its adapter and normalizer modules. The canonical record has twelve fields: `id`, `source`, `source_id`, `title`, `abstract`, `authors[]`, `year`,

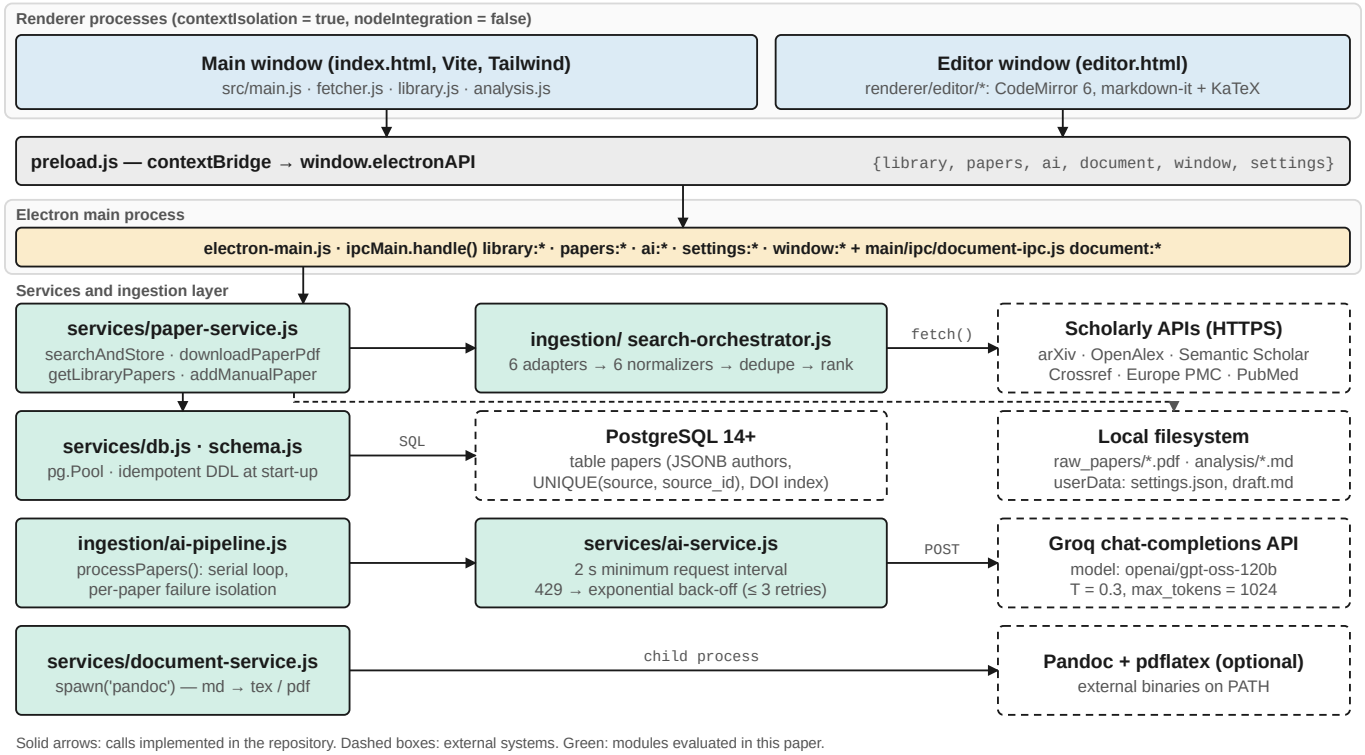


Fig. 1. **[C]** Architecture of RIE as implemented in the repository (commit `bf2d3a3`). Renderer processes reach services only through the `contextBridge` API and IPC handlers. The dashed arrow from `paper-service.js` denotes file I/O to `raw_papers/` (PDF library). Green boxes are the modules exercised by the experiments in Section VI; the editor and export path are described but not evaluated.

TABLE I
[C] PROVIDER ADAPTERS AS IMPLEMENTED (`INGESTION/SOURCES/*.JS`). "REQ." IS THE NUMBER OF UPSTREAM HTTP REQUESTS PER SEARCH FOR LIMIT k. TIMEOUTS ARE THE ADAPTER'S `ABORTCONTROLLER` VALUES.

Provider	Req.	Timeout	Behaviour noted in code
OpenAlex	1	15 s	$3k$ candidates, relevance sort; optional key
arXiv	1	30 s	$3k$ candidates; `ti:` OR `abs:`; $\geq$3 s between calls (module-wide)
Semantic Scholar	1(+1)	15 s	one retry after 2 s on HTTP 429; optional key
Crossref	1	15 s	scholarly-type filter; `mailto`
Europe PMC	$1{+}n$	15 s	Crossref look-up per hit with a DOI ($n \leq k$; 3 s timeout) for a PDF link
PubMed	2	25 s	ESearch then EFetch (XML), 350 ms spacing; optional key

`venue`, `doi`, `url`, `pdf_url`, `citation_count`. Provider-specific handling visible in the code (Table I) includes reconstruction of OpenAlex abstracts from an inverted index, stripping of JATS markup from Crossref abstracts, a Crossref look-up for each Europe PMC hit that carries a DOI, to find a direct PDF link, and deliberate omission of PubMed PDF links (documented in the code as blocked by an anti-bot challenge).

C. Search pipeline

A call `searchAndStore(q, sources, k, sortBy)` (Figure 2) validates source names, dispatches all adapters concurrently with `Promise.allSettled`, normalizes each provider's raw items, ranks them and keeps the top k per provider, deduplicates the union, assigns `randomUUID` identifiers *after* deduplication, ranks the union again, and inserts the records into PostgreSQL. Ranking is by publication year then citation count (`sortBy=date`) or the reverse (`sortBy=citations`); records with missing values sort last. Provider relevance order is therefore *not* preserved: for `date`, the candidate pool is the $3k$ most relevant OpenAlex results and the $3k$ most recent arXiv results, which are then re-sorted by year. The orchestrator returns the ranked list plus `providerStats` and `providerFailures` for display.

D. Persistence

A single table `papers` stores the canonical record plus `created_at` and `local_pdf_path`; `authors` is JSONB; `UNIQUE(source, source_id)`, a unique partial index on `doi`, and an index on `created_at` are created idempotently at start-up. Insertion is one `INSERT ... ON CONFLICT DO NOTHING` per record followed by a `SELECT` that finds the surviving row (by `source, source_id` or DOI) and overwrites the in-memory `id` with the database identifier, so repeated searches return stable identifiers. The README additionally

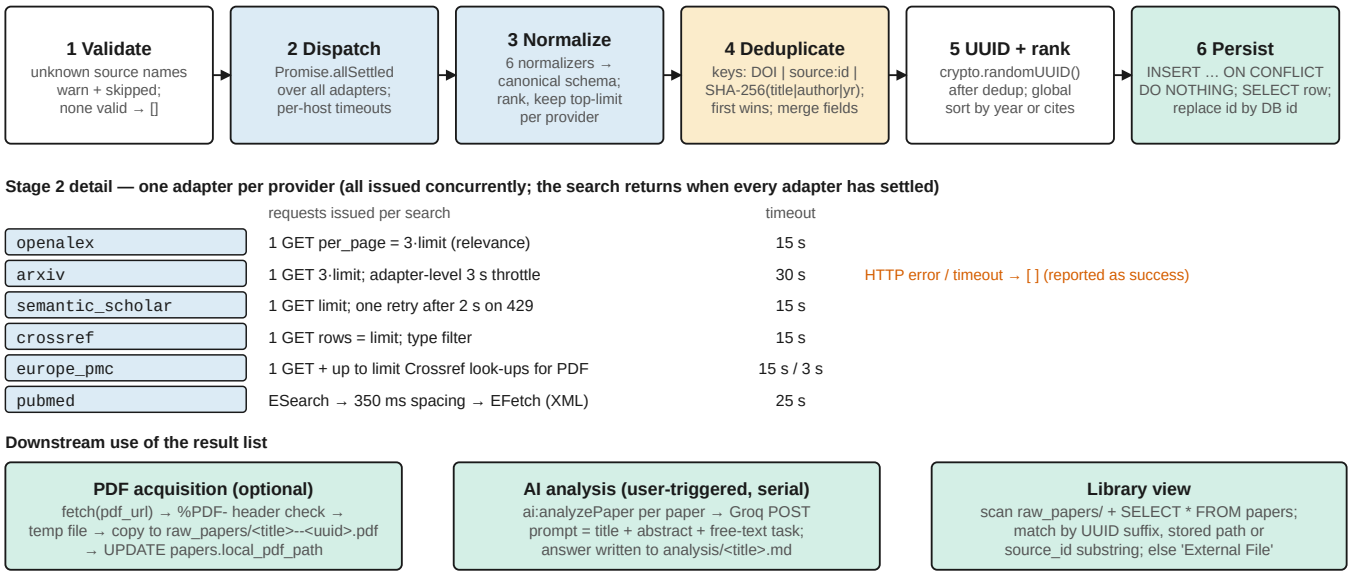


Fig. 2. **[C]** Data flow of a search (stages 1–6), the per-provider requests of stage 2, and downstream consumers. The orange annotation marks a behaviour established experimentally [S] in Section VI: an arXiv adapter failure is swallowed and appears as a successful, empty provider.

mentions `analyses` and `collections` tables and "indexed full-text fields"; the schema module creates neither (Section V).

E. PDF acquisition, library and AI analysis

`downloadPaperPdf` fetches a PDF, rejects `text/html` responses and bodies lacking the `%PDF-` magic bytes, writes to a temporary file outside the library, copies it to `raw_papers/<title>--<uuid>.pdf`, and records a workspace-relative path. `getLibraryPapers` lists `raw_papers/`, loads *all* rows of `papers`, and associates files with rows by UUID suffix, stored path, or a substring test on `source_id`; unmatched files are shown as "External Document". AI analysis is user-triggered and serial: for each paper the renderer calls `ai:analyzePaper`, which POSTs a chat-completion request (system prompt, then a user message containing title, abstract and a free-text task; temperature 0.3, `max_tokens` 1024) to the Groq endpoint with a minimum interval between request starts (default 2 s) and exponential back-off on HTTP 429. Each answer is written to `analysis/<title>.md`.

IV. Methodology

A. Formal model of the search pipeline

Let $\mathcal{S}$ be the selected providers and T_s the time adapter s needs to *settle* (resolve or reject). Because the orchestrator awaits `Promise.allSettled`, the time to first result is

$$T_{\text{search}} = \max_{s \in \mathcal{S}} T_s + \varepsilon_{\text{local}}, \tag{1}$$

where $\varepsilon_{\text{local}}$ is the local normalize–deduplicate–rank cost. For adapters with dependent requests, and a per-request service time L, the code implies

$$\begin{aligned} T_{\text{oa}} = T_{\text{s2}} = T_{\text{cr}} = L, \quad T_{\text{ax}} = w_{\text{ax}} + L, \\ T_{\text{epmc}} = L + \max_{i \le k} L_i^{\text{cr}} \approx 2L, \\ T_{\text{pm}} = \max(L, \rho) + L, \quad \rho = 0.35\,\text{s}, \end{aligned} \tag{2}$$

where $w_{\text{ax}} = \max(0,\, 3\,\text{s} - \Delta)$ is the arXiv throttle wait given Δ seconds since the previous arXiv call started (state is module-global, so it persists across searches), and the Europe PMC enrichment look-ups run concurrently. Equation (2) assumes a cold start: the PubMed adapter's spacing state is also module-global, so a search begun less than ρ after the previous PubMed request would wait for its first request as well [C]. With a uniform L this gives $T_{\text{search}} \approx \max(2L,\ \max(L, \rho) + L)$; at $L{=}0$ the PubMed spacing alone sets a floor of ρ. An upper bound on the worst case follows from the timeouts in Table I: $T_{\text{search}} \lesssim \max(3{+}30,\ 25,\ 15{+}3,\ 2{+}2{\cdot}15) \approx 33\,\text{s}$, attained when an adapter hangs. The upstream request count for a search is $N_{\text{req}} = 1{+}1{+}1{+}1{+}(1{+}n_{\text{epmc}}){+}2 = 7 + n_{\text{epmc}}$ where $n_{\text{epmc}} \le k$ is the number of Europe PMC hits that carry a DOI.

B. Deduplication as implemented

For a canonical record p define three keys: $\kappa_{\text{doi}}(p)$ (the lower-cased DOI without resolver prefix, if any), $\kappa_{\text{src}}(p) = (\text{source}, \text{source_id})$ (if `source_id` is non-empty), and $\kappa_h(p) = \text{SHA256}(\text{lc}(\text{title}) \,|\, \text{lc}(\text{first author}) \,|\, \text{year})$. Records are processed in input order; a record is a duplicate of the *first* retained record that has an equal key (tested in the order DOI, source, hash), in which case missing `pdf_url`, `abstract`, `venue`, `doi` and `citation_count` of the retained record are filled from the duplicate. Otherwise the record is retained and *its*

keys are registered. Two properties follow directly from the code and are tested experimentally.

Property 1 (order dependence). A duplicate's keys are not registered, even when they differ from the retained record's. Consider records a, b, c of one work with $\kappa_h(a) = \kappa_h(b) \neq \kappa_h(c)$, a without DOI, and $\kappa_{\text{doi}}(b) = \kappa_{\text{doi}}(c)$. For input order (a, b, c): b merges into a via κ_h, b's DOI is not registered, so c matches nothing and two records are output; for order (b, a, c), a merges into b via κ_h and c merges via the DOI, giving one record. The result thus depends on provider order and per-provider ranking, i.e., the procedure is not confluent. A transitive-closure (union-find) resolution over the same keys is order independent.

Property 2 (no negative DOI evidence). The lookup chain evaluates κ_h even if both records carry DOIs that differ, so differing DOIs never prevent a merge.

Cost. Each record costs $O(1)$ hash-table operations plus one SHA-256, so the procedure is $O(n)$.

C. Expected residual duplicates

For a same-work pair in which at least one record lacks a DOI (typical for arXiv, Table I) and the source identifiers differ, a merge happens only if the strict keys coincide. Writing p_t, p_a, p_y for the probabilities that the normalized title, first-author string and year agree, and *assuming independence*, $\Pr[\text{merge}] \approx p_t p_a p_y$ and the expected number of residual duplicates among M such pairs is $M(1 - p_t p_a p_y)$. We do not estimate p_t, p_a, p_y for real providers; Section VI instead measures the response of the implementation to controlled values of each factor.

D. AI-analysis scheduling

With minimum start interval τ (default 2 s) and per-request latency L_i, the loop waits $\max(0, \tau - L_{i-1})$ after request $i-1$ returns, so starts are separated by $\max(L_{i-1}, \tau)$ and a batch of N papers takes $T_N = \sum_{i<N} \max(L_i, \tau) + L_N$ plus retries. On HTTP 429 the a-th retry waits $2^a \text{ s} + U[0, 0.5)$ s for $a \leq m$ (default $m = 3$), so $k \leq m$ consecutive 429s add $\sum_{a=1}^{k}(2^a + 0.25)$ s in expectation; the $(m+1)$-th consecutive 429 raises the error.

E. Experimental design

Harness. Experiments run the repository modules unmodified under Node.js; the only substitution is `globalThis.fetch`, replaced by an emulator that (i) routes by host to one of six provider mimics or the Groq endpoint, (ii) renders records of a synthetic ground-truth corpus into the raw response shape that the corresponding normalizer reads (derived from the normalizers and the repository's own test fixtures, *not* from recorded live traffic), (iii) injects a configurable latency, and (iv) injects faults (HTTP 500/429, network error, hang until the adapter's abort signal). Database experiments use a real PostgreSQL 16.15 server on the loopback interface with the repository's schema module. Every random choice is seeded. Two experiments call library functions directly rather than through the orchestrator: part D of E2 replicates orchestrator steps 1–3 by calling the same adapters, normalizers and ranker (its output was checked against the real orchestrator), and E1 times `xml2js` parsing with the adapters' parser options. A separate script (`verification/verify_module_usage.mjs`) confirms that no tracked application file differs from the baseline commit, that the replicated pipeline returns the same record set *and* ranked order as the real orchestrator for three further queries, and that the synthetic renderings survive a round trip through the real normalizers (200 of 200 records agree with ground truth in every field for every provider, except the first-author string for Europe PMC, which is rendered as "Surname Initial" by design).

Synthetic corpus. Works have unique titles (a numeric token guarantees no accidental collisions), 2–6 authors including names with diacritics, a DOI, a PMID, an arXiv identifier for about half the works, and a preprint year equal or one year earlier than the journal year. Because identity is known, merge outcomes are scored exactly. *The corpus says nothing about real overlap rates.*

Metrics. *Pairwise merge recall* is the fraction of same-work record pairs collapsed to one record. *False merges* count distinct-work pairs collapsed. *Residual duplicates* are $|\text{output}| - |\text{distinct works}|$. Timings use `performance.now()`; we report medians (with quartiles where $n \geq 5$), discarding warm-up repetitions. Latency in the emulator is an injected parameter; measured wall-clock is compared with the models of the previous subsections.

Platform [S]. 4 vCPU Intel Xeon 2.3 GHz, 16 GB RAM, Ubuntu 24.04.5, Node.js v22.23.4, PostgreSQL 16.15, `pg` 8.20.0, `xml2js` 0.6.2 (recorded by `experiments/env_info.mjs`); a single cloud machine, with the Electron shell and GUI not run. Experiments that measure wall-clock time were run with no other load.
Variants. Besides the repository's `deduplicate()`, two harness-only variants are evaluated: UF-REPO (union-find over the same three keys) and UF-RELAXED (union-find over DOI, `source:id`, and a relaxed key: diacritic-folded alphanumeric title, first-author surname and first initial, linked only if years differ by at most one). The relaxed key is a prototype for quantifying trade-offs, not a proposal evaluated on real data.

F. Evidence classes and the live session

To keep simulated and live evidence apart, every result is assigned one of four classes (Table II); table and figure captions carry the label. The **live session [L]** is a set of observations that the author recorded from the Windows desktop build of RIE using its own in-app logs: one query ("transformers"), one session, no repetitions. We report these observations exactly as recorded, derive only arithmetic quantities from them (e.g., a mean per paper), and state which quantities were *not* recorded. They were not reproduced in the cloud, are not benchmarks, and are used only to check whether the emulated findings are *qualitatively* consistent with the real application. The AI timing covers analysis and saving of reports, not summary accuracy, and the editor timing is an approximate observation.

TABLE II
EVIDENCE CLASSES USED IN THIS PAPER.

Class	Definition	Reproducible here?
[S]	Unmodified RIE modules against an emulated upstream (and real local PostgreSQL); Section VI-B to -E	Yes (`run_all.sh`)
[R]	Repository test scripts executed in the cloud sandbox; Section V	Yes, with network access
[L]	One user-recorded session of the real Windows application, from in-app logs; Section VI-F	No; not independently reproduced
[C]	Code inspection with file and line evidence; not executed	Not applicable

TABLE III
[C] SIZE OF THE CODE BASE BY LAYER (PHYSICAL LINES).

Layer	Files	Lines
Ingestion: orchestration (orchestrator, dedupe, ranking, ai-pipeline)	4	271
Ingestion: normalizers	6	720
Ingestion: source adapters	6	454
Services (db, schema, paper, ai, document)	5	746
Electron main, IPC, preload	3	431
Renderer: main-window modules	5	1 559
Renderer: editor modules	3	477
HTML views	2	1 057
Tests	4	772
Total	38	6 487

V. IMPLEMENTATION AND REPOSITORY AUDIT

Technology. RIE is a Node.js ES-module code base (Electron 40 shell, Vite 5 and Tailwind 3 front end, CodeMirror 6, `markdown-it` with KaTeX, `pg` 8 for PostgreSQL, `xml2js` for arXiv and PubMed XML). Provider calls use the platform `fetch` with per-adapter `AbortController` timeouts; the orchestrator isolates provider failures with `Promise.allSettled`; normalizers are total functions. Table III gives size per layer (physical lines of tracked files at commit `bf2d3a3`, produced by `e0_repo_stats.mjs`).

A. What is implemented, validated, and documented but absent

Table IV separates implemented functionality from validated functionality, using (a) the four test scripts in the repository, which we executed in the sandbox [R], (b) the emulated experiments of this paper [S], and (c) the single live session [L]. We executed the four test scripts twice with outbound network access [R]. Because the sandbox's egress proxy carries only HTTPS, Node's `fetch` was routed through it (`NODE_USE_ENV_PROXY=1`) and a preload shim (`experiments/https_shim.mjs`) upgraded the plain-HTTP arXiv URL used by `arxiv.js` to HTTPS; no test or application file was changed. In both runs all 78 assertions passed (36 in `test-all-providers.js`, 8 in `test-ingestion.js`, 14 in `test-paper-service.js`, 20 in `test-workspace-library.js`). No API keys were configured, and OpenAlex and Semantic Scholar answered HTTP 429 in both runs; the tests tolerate this, so a green suite does not imply that every provider responded. (An earlier execution without network access is preserved in `logs/`; there only the 57 assertions that do not need live providers could be evaluated.) The offline unit tests cover each normalizer with one hand-written fixture. Several assertions in `test-ingestion.js` are upper bounds (e.g., "at most 6 results") and would also pass if every provider returned nothing; they do not by themselves demonstrate retrieval. There is no test of the AI service and one test of deduplication (a single pair that shares a DOI).

Documentation drift. The README (i) lists arXiv and OpenAlex as the only sources whereas six are registered, (ii) names `llama-3.3-70b-versatile` whereas the code default is `openai/gpt-oss-120b` (git history records the change), (iii) lists an `orchestrator.js` file whereas the module is `search-orchestrator.js`, (iv) states that tables `analyses` and `collections` are created and that full-text fields are indexed, which `schema.js` does not do, and (v) the header comment of `src/analysis.js` still refers to another LLM vendor. No roadmap or issue list exists in the repository, so we cannot enumerate "planned" work beyond the features that the README describes and the code lacks (row 10 of Table IV).

VI. EXPERIMENTAL EVALUATION

A. Setup and experiment inventory

All [S] measurements below were produced by `experiments/run_all.sh` (raw JSON in `results/`; every quoted number is re-derived from those files by `verification/verify_claims.mjs`, and an independent re-run is compared in `verification/`). Upstream services are emulated; injected latencies are parameters. Medians are over the repetitions in Table V, after discarding warm-up repetitions. A fixed seed governs every random choice. **Repeatability.** The whole sweep was executed twice, the second time independently from a clean copy of the harness. Counts, statuses and outputs (all of E2, request counts, fault statuses, database row counts, AI request counts and prompts) were identical; wall-clock results dominated by injected delays agreed within about 15 ms; sub-10 ms CPU timings did not always agree (two of 75 compared quantities fell outside the stated tolerance, both CPU microtimings; see `verification/claims_report.md`).

B. RQ1: search latency, throttles and failure containment [S]

Latency structure. We issued a six-provider search (k=10, each run after $\geq$3.3 s of idleness so that the arXiv throttle is not engaged) with the same injected latency L for every upstream request. Figure 3 shows that the observed median equals the parallel model of Equations (1) and (2) to within 4–11 ms at every L (e.g., 2 011 ms observed vs. 2 000 ms predicted at L=1 s), and is far below the sum-of-adapters and fully serial alternatives (8 000 and 17 000 ms at L=1 s). At L=0 the search still takes $\approx$358 ms: the 350 ms spacing between PubMed's two requests is a floor that no upstream speed can remove.

TABLE IV
IMPLEMENTATION STATUS. "EVIDENCE" LISTS THE CLASSES OF TABLE II THAT SUPPORT EACH CAPABILITY: [R] REPOSITORY TESTS RUN IN THE SANDBOX, [S] EMULATED-UPSTREAM EXPERIMENTS, [L] THE SINGLE LIVE SESSION, [C] CODE INSPECTION ONLY.

Capability	Implemented	Evidence	Remark
Concurrent retrieval from 6 providers	yes	[R], [S], [L]	Live: four providers returned results, Semantic Scholar HTTP 429; live latency not recorded; the repository's live-provider tests passed in two online runs [R] (OpenAlex and Semantic Scholar returned HTTP 429 without API keys)
Normalization to canonical schema	yes	[R], [S]	One hand-written fixture per provider; no recorded live payloads
Deduplication (3 keys)	yes	[R], [S]	Order dependent; fails under single-factor metadata disagreement; one repository test (one DOI pair)
Ranking (year / citations)	yes	[S], [C]	Provider relevance order is discarded; retrieval quality not evaluated
PostgreSQL persistence, stable IDs	yes	[R], [S], [L]	Live: initialization succeeded and papers appeared in the Library; sequential per-record INSERT+SELECT
PDF download, library indexing	yes	[R], [L]	20/20 repository assertions; live: retrieval worked for OpenAlex and arXiv records
AI analysis (Groq, serial, retry)	yes	[S], [L]	Live: 8 papers analyzed and saved in 27 s; scheduling and retry emulated; output accuracy, cost and provider rate limits not assessed
"Structured synthesis", "key contribution extraction" (README)	partly	[C]	Only a free-text prompt is sent; no output schema
Markdown/LaTeX editor, Pandoc export	yes	[L] (start-up only)	Editor opened in ≈1–2 s (approximate); editing, rendering and Pandoc export not evaluated
`analyses`, `collections` tables; full-text indexes (README)	no	[C]	Schema creates only `papers`

TABLE V
EXPERIMENT INVENTORY (ALL CLASS [S] UNLESS NOTED). "REPS" ARE TIMED REPETITIONS AFTER WARM-UP.

ID	Measures	Sample / repetitions
E1	normalizer and XML cost	2 000 records × 9 passes; XML 20 reps
E2	deduplication (A–D)	161–500 works per probe; 1 000 works × 3 records × 6 orders; 20 queries × 2 limits
E2b	dedup scaling	$n \leq 10^5$; 7 reps (10^5: 3)
E3	orchestrator: sweep, warm start, amplification, faults, hang, CPU	5 latencies × 5 reps; 4 gaps × 5; 1 run per k; 13 fault cases; 3 hangs × 3; 5 limits × 15
E4	PostgreSQL layer	4 limits × 6; one 8-query session; 12 library sizes × 5
E5	AI client (emulated endpoint)	4 latencies × 1 run of 6 papers; 4 back-off cases × 3; 4 error codes; prompt cases
L	live session [L]	1 query, 1 session, no repetition

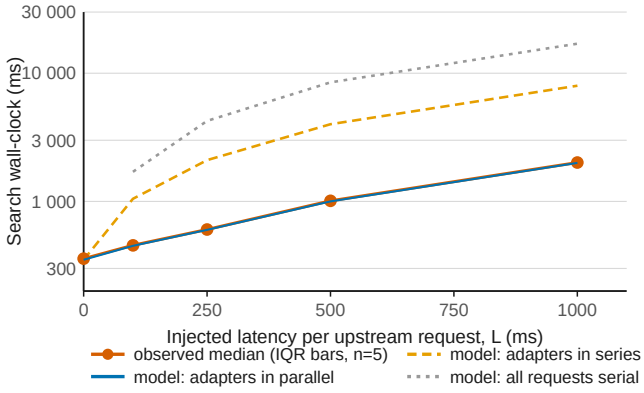


Fig. 3. **[S]** Search wall-clock vs. injected per-request latency (six providers, k=10, cold start). Observed medians (IQR bars; invisible at this scale) coincide with the parallel model; serial alternatives are shown for contrast.

Interquartile ranges were about 1 ms or less, so the emulator-driven runs are highly repeatable at that precision; this says nothing about real network jitter.

Back-to-back searches. The arXiv adapter's 3 s spacing is module-global state. With L=100 ms, a second search started immediately after the first returned took 2 652 ms (range 2 642–2 656) instead of 455 ms; after idle gaps of 1 s and 2 s it took 1 650 and 647 ms; after 3.5 s it took 454 ms, in line with $w_{\text{ax}} = \max(0, 3 - \Delta)$. A user who refines a query within about three seconds therefore waits for the throttle whenever arXiv is selected, because the wait is applied inside the adapter.

Request amplification. Table VI lists upstream requests per search: $N_{\text{req}} = 7 + k$, because Europe PMC issues one Crossref look-up for every returned hit that carries a DOI (every emulated hit does), all concurrently and without a bound. At k=50 a single search emits 57 HTTP requests, 50 of them to a host that is also a selected provider.

TABLE VI
[S] UPSTREAM HTTP REQUESTS PER SIX-PROVIDER SEARCH.

Limit k	5	10	20	50
Total requests	12	17	27	57
of which Crossref look-ups for Europe PMC	5	10	20	50

Faults. We injected faults into one provider at a time (Table VII). For five providers an HTTP 500 or network error is reported correctly (`status: failed`, listed in `providerFailures`) and the other providers' results are returned unchanged within the normal time. The arXiv adapter, however, catches every error and returns `[]`, so

TABLE VII
[S] FAULT INJECTION INTO ONE PROVIDER (OTHER FIVE HEALTHY). "REPORTED" IS THE STATUS IN `providerStats`; "LISTED" = APPEARS IN `providerFailures`. THE 429 CASE INCLUDES ONE 2 S RETRY.

Fault	Provider	Reported	Listed	Time
HTTP 500 / net. error	OpenAlex, S2, Crossref, EPMC, PubMed	failed	yes	0.11–0.41 s
HTTP 500 / net. error	arXiv	success, 0 items	**no**	0.41 s
HTTP 429	Semantic Scholar	failed	yes	2.11 s
Hang	OpenAlex	failed	yes	15.02 s
Hang	PubMed	failed	yes	25.03 s
Hang	arXiv	success, 0 items	**no**	30.03 s

the orchestrator records `status: success, raw: 0` and nothing is listed as failed; the orchestrator's own status output therefore cannot distinguish an arXiv outage from a query with no arXiv hits. The same masking appears in the repository's own `test-paper-service.js` log in the sandbox ("`[source=arxiv] failed: fetch failed`" followed by a successful, empty summary). A hung provider stalls the entire result set until its adapter timeout (15.0, 25.0 and 30.0 s for OpenAlex, PubMed and arXiv; n=3 each, spread below 3 ms), since the orchestrator has no overall deadline; this matches the worst-case bound of Section IV.

C. RQ2: deduplication [S]

Single-factor disagreement. For up to 500 works we built two records of the same work, one from an OpenAlex-style rendering (with DOI) and one from a record that lacks a DOI (arXiv-style, via the real arXiv normalizer), differing in one metadata factor at a time; Figure 4 reports the fraction merged. When both records carry the same DOI, every variant merges all pairs for every perturbation (recall 1.00; not plotted). Without a shared DOI the repository's `deduplicate()` merges only metadata-identical records and records differing in letter case: a one-year difference in publication year, any change of first-author format ("A. Surname", "Surname A"), a trailing period, a changed subtitle separator, a dropped subtitle, diacritic folding of the first author, and reversed author order each give recall 0.00. The UF-REPO variant behaves identically on these pairs, as expected, since it uses the same keys. The relaxed-key prototype restores recall 1.00 for all but two probes, reversed authors (the first-author key changes) and a dropped subtitle (the title key changes), which require different evidence.

Order dependence. Table VIII shows the three-record scenario of Property 1 for 1 000 works (a DOI-less record, a DOI record with identical strings, a DOI record with different strings) under all six input orders. `deduplicate()` outputs 1 000 records for two orders and 2 000 for four, i.e., in four of the six orders every work is output twice, depending only on which provider is listed first; both union-find variants output the 1 000 correct records for every order. Whether a real search lands in a favourable or unfavourable order depends on provider selection and per-provider ranking, not on anything the user controls directly.

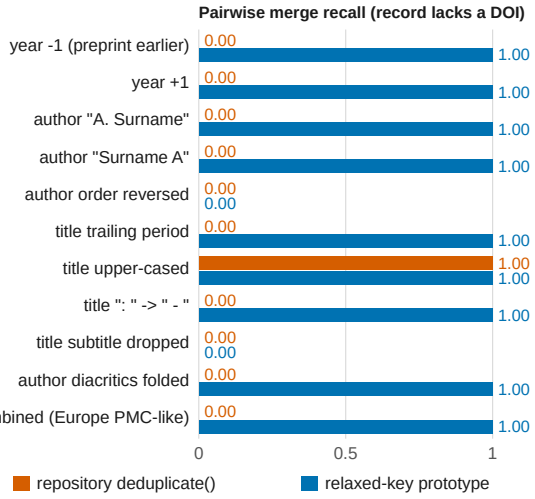


Fig. 4. **[S]** Pairwise merge recall for a same-work pair where one record lacks a DOI, by metadata perturbation (n=161–500 works each; synthetic corpus). Case-only changes are handled by the repository's lower-casing.

TABLE VIII
[S] OUTPUT RECORDS FOR 1 000 WORKS × 3 RECORDS UNDER EACH INPUT ORDER (TRUTH: 1 000). A: NO DOI; B: DOI, STRINGS IDENTICAL TO A; C: DOI, DIFFERENT STRINGS.

Input order	ABC	ACB	BAC	BCA	CAB	CBA
`deduplicate()`	2000	2000	1000	1000	2000	2000
UF-REPO	1000	1000	1000	1000	1000	1000
UF-RELAXED	1000	1000	1000	1000	1000	1000

False-merge probes. (i) 500 pairs of distinct works with different DOIs but identical title, first author and year were all merged by all variants (Property 2). (ii) 100 records with empty title and no authors but distinct DOIs and a common year collapsed to a single record in every variant. (iii) 500 pairs with the same title and surname and the same first initial but different given names ("Wei" vs. "Wen" Zhang) were never merged by the repository (0/500) but always merged by the relaxed prototype (500/500). The last result is the cost of the relaxation, and is why the prototype is not offered as a drop-in fix.

End-to-end mixed scenario. We ran the real adapters and normalizers through the emulator for 20 queries per setting, over 400 works with provider membership probability 0.5, abbreviated Europe PMC author strings, and a 40 % preprint-year skew for arXiv-bearing works (Table IX). Because the orchestrator's own pre-deduplication list is not exposed, the harness replicates its steps 1–3 with the same modules; the replicated pipeline and the real orchestrator produced identical output sizes (58 and 133 records for the checked queries; the verification script additionally compared record sets and order). `deduplicate()` left 0.8 % (9 of 1 121 works) and 2.6 % (66 of 2 522 works) of the works duplicated at k=10 and k=25, all involving an arXiv record (arXiv with OpenAlex, Europe PMC, Crossref, PubMed or Semantic Scholar); union-find over the same keys removed none of the 9 at k=10 and only 4 of

TABLE IX
[S] MIXED END-TO-END SCENARIO (20 QUERIES, EMULATED PROVIDERS, SYNTHETIC CORPUS). RESIDUAL = OUTPUT RECORDS MINUS DISTINCT WORKS.

	k=10		k=25	
Variant	Output	Residual	Output	Residual
Raw candidates (distinct works)	1 200 (1 121)		3 000 (2 522)	
`deduplicate()`	1 130	9	2 588	66
UF-REPO	1 130	9	2 584	62
UF-RELAXED	1 121	0	2 522	0

TABLE X
[S] `SEARCHANDSTORE` COST SPLIT (FOUR JSON PROVIDERS, LOCAL POSTGRESQL; MEDIANS, n=6).

k	Papers	Orchestrator	Total (cold)	Repeat	DB ms/paper
10	40	5.1	33.4	17.7	0.71
25	97	7.5	67.5	31.9	0.62
50	191	10.6	118.3	60.3	0.56
100	355	19.6	207.2	109.9	0.53

the 66 at k=25; the relaxed prototype removed all residual duplicates with no work lost. These rates are driven by the assumed skew and author format and are illustrative only; what the experiment shows is that the mechanism operates through the full pipeline.

D. RQ3: local cost and scaling [S]

Normalization and parsing. Normalizers cost between 3.1 and 6.6 µs per record, and parsing an arXiv Atom response of 100 entries (79 kB) or a PubMed EFetch response of 100 entries (122 kB) took 2.4–3.8 ms and 5.7–7.4 ms respectively (ranges over two independent executions of the full sweep, Section VI-A). Microsecond-level timings were the least stable measurements in the study: individual providers differed by up to 45 % between the two executions (e.g., OpenAlex 6.6 vs. 3.6 µs), so we make no ranking claim among providers and treat these figures as orders of magnitude. Orchestration of four JSON providers (fetch emulation, parse, normalize, rank, deduplicate, UUIDs) took about 5 ms at k=10, about 19 ms at k=100 and 32–43 ms at k=200 (medians, n=15 per setting, two executions), growing about linearly with the 40–800 candidates. Local compute is thus negligible next to the adapter timeouts of 15–30 s and to any injected latency we used.

Persistence. On a local PostgreSQL 16 instance (Table X), `searchAndStore` adds about 0.5–0.7 ms per returned paper for the sequential `INSERT` plus `SELECT` (two round trips per record), i.e., 28 ms for 40 papers and 188 ms for 355. Repeating the identical search (all rows conflict) costs roughly half, returns identical identifiers (checked for all four limits), and leaves the row count unchanged, confirming that the `ON CONFLICT DO NOTHING` + re-select design provides stable identifiers. Over a network the two round trips per record would dominate; we did not measure this.

Persisted duplicates. Over a session of eight overlapping six-provider queries (k=20, 300-work corpus), the table held 310 rows for 271 distinct works: 39 rows (12.6 %) were persisted duplicates. The unique DOI index collapses duplicates that share a DOI; the residual ones must involve the 45 rows without a DOI, which are arXiv records (47 arXiv rows, two of which had acquired a DOI through the in-memory merge). In this setting the deduplicator's DOI-less weakness (RQ2) therefore carries into the permanent library, where nothing later reconciles it.

Dedup scaling and library scan. Figure 5(a) shows that `deduplicate()` scales roughly linearly: 2.3 ms for 1 000 records, 28 ms for 10 000 and 626 ms for 100 000 (including shallow copies); the union-find variants have the same order of cost, so order independence costs essentially nothing. Figure 5(b) shows that `getLibraryPapers()` loads the full table on every call and, for each PDF without a UUID or path match, scans all rows with a substring test: with unmatched files its time grows with the product of rows and files (about 47–52 ns per row-file pair), from 4 ms (1 000 rows, no files) to 51 ms (1 000 rows, 1 000 files) and 1.10 s (20 000 rows, 1 000 files); even with no files, loading 20 000 rows takes 48 ms.

E. RQ4: AI-analysis client [S]

Scheduling. With the shipped 2 s interval, six papers took 10.2, 11.0, 12.0 and 18.0 s for injected model latencies of 0.2, 1, 2 and 3 s; the model $5\max(L,\tau)+L$ predicts 10.2, 11.0, 12.0 and 18.0 s (Figure 6a), and start-to-start gaps had a median of 2 002, 2 001, 2 003 and 3 004 ms. The loop is strictly serial, so throughput is bounded by $\max(L,\tau)$ per paper regardless of what the provider would allow.

Retry behaviour. Success after one, two or three consecutive 429 responses took 2.2–2.3, 6.2–6.7 and 14.4–14.8 s respectively (n=3 each), matching $\sum(2^a+0.25)$ s; with four consecutive 429s the client issued four requests in 14.7–14.9 s and then threw (Figure 6b). HTTP 400, 401, 500 and 503 responses were *not* retried (one request each), so transient server errors fail a paper immediately; `Retry-After` is ignored. Inside `processPapers`, an injected failure at the third of five papers was isolated (4 succeeded, 1 failed, the failure recorded with its title).

Prompt construction. The user message is `Paper Title: $title\nAbstract: $abstract\n\nTask: $prompt` with no null handling. For a paper whose abstract is `null` (the normalizers' value for a missing abstract; arXiv, PubMed and Crossref records can lack one) the model receives the literal text "`Abstract: null`", for an absent field "`Abstract: undefined`", and a 20 000-word abstract is sent whole (100 060 characters) with no truncation. Responses are requested with `temperature` 0.3 and `max_tokens` 1024 for model `openai/gpt-oss-120b`. Whether a model given "`Abstract: null`" fabricates content is not measured here; the prompt gives it no instruction to refuse.

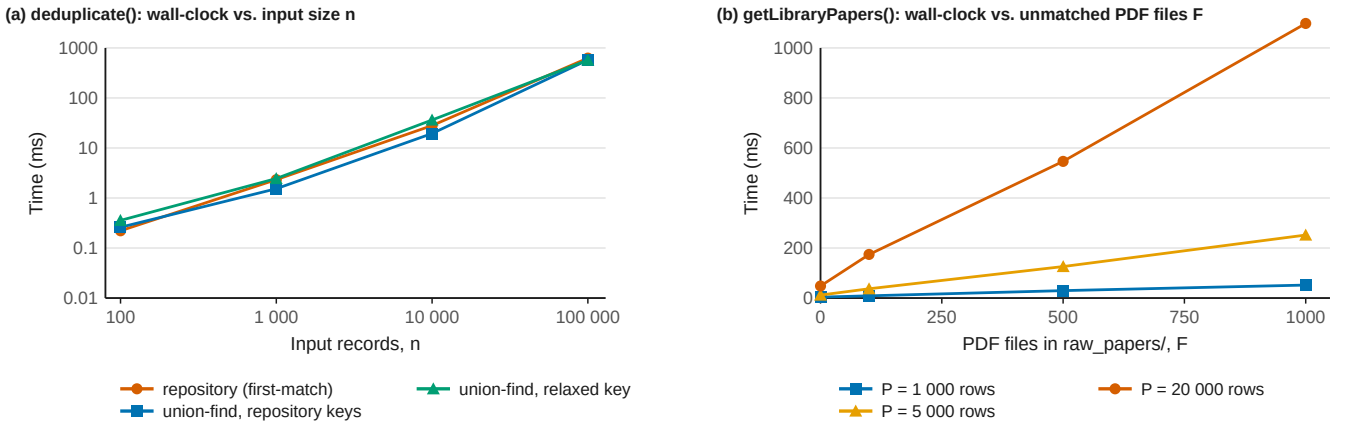


Fig. 5. **[S]** Scaling of local components. (a) `deduplicate()` and two union-find variants versus input size (log–log; medians). (b) `getLibraryPapers()` for unmatched PDF files, for three database sizes P (medians, n=5).

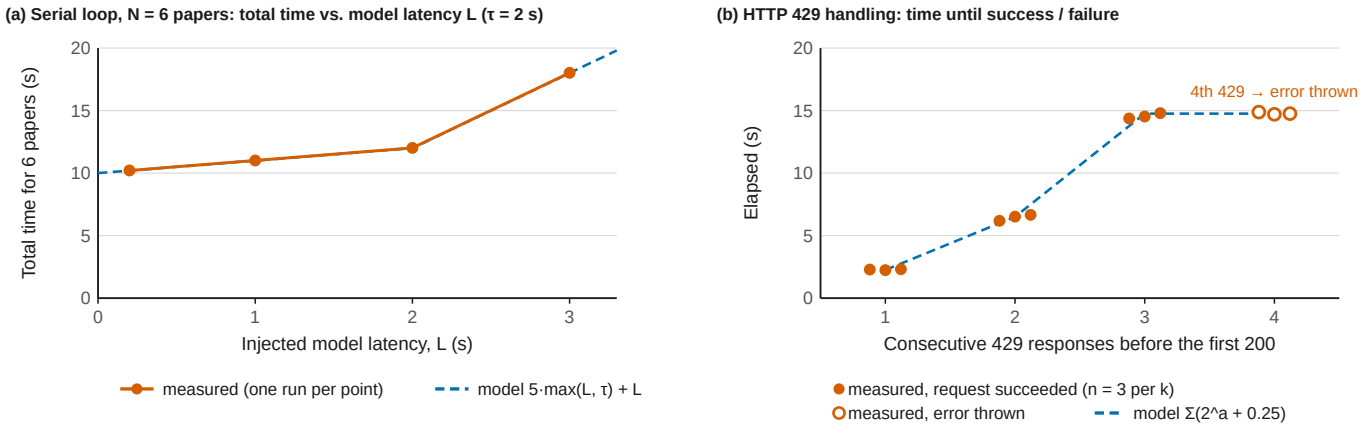


Fig. 6. **[S]** AI client scheduling against an emulated endpoint. (a) Total time for six papers vs. model latency; the first request is not delayed. (b) Elapsed time for k consecutive HTTP 429 responses followed by success; at k=4 the retry budget is exhausted and the error is thrown.

F. Live observations from the Windows desktop application [L]

The emulated experiments cannot show that the real application works against the real services. The author therefore ran RIE on a Windows desktop with the real providers and the Groq endpoint and recorded the observations in Table XI from the application's in-app logs. This is class [L] evidence: one query, one session, no repetition, not reproduced in the cloud, and not mixed with any [S] result.

Not recorded. The configured per-provider limit and sort order, whether PubMed was selected, the number and size of downloaded PDFs, search latency, per-paper AI latencies, whether Semantic Scholar's built-in retry fired, the model identifier (the code default is `openai/gpt-oss-120b`, overridable by `GROQ_MODEL`), the machine and network, and the Windows and Electron versions. No retrieval-relevance or summary-accuracy assessment was made: the AI timing measures processing and saving only.

Comparison with the emulated findings. Table XII sets each live observation next to the emulated characterization it can be compared with. Where no comparison is possible we say so.

TABLE XI
USER-RECORDED LIVE OBSERVATIONS [L] (WINDOWS DESKTOP, QUERY "TRANSFORMERS"). THE MEAN PER PAPER IS DERIVED AS $27/8$; EVERYTHING ELSE IS AS RECORDED.

Item	Recorded observation
Search query	"transformers"
Providers returning results	OpenAlex, arXiv, Crossref and Europe PMC: 2 papers each, 8 papers in total
Semantic Scholar	HTTP 429 (rate limit encountered)
AI analysis	Groq pipeline analyzed all 8 papers and saved a Markdown report for each: 27 s in total
Mean time per paper	$27/8 \approx 3.4$ s (derived; covers analysis and saving)
PostgreSQL	initialized successfully; retrieved papers appeared in the Library
PDF retrieval	worked for OpenAlex and arXiv records
In-app editor	opened in approximately 1–2 s (approximate observation)

Interpretation. The live session shows that, on one real

TABLE XII
EMULATED FINDINGS [S] VERSUS THE SINGLE LIVE SESSION [L]. "RELATIONSHIP" STATES HOW FAR THE LIVE OBSERVATION SUPPORTS THE EMULATED FINDING; IT IS NOT A QUANTITATIVE VALIDATION.

Aspect	Emulated finding [S]	Live observation [L]	Relationship
Provider failure containment	An injected HTTP 429 on Semantic Scholar is reported as failed after one 2 s retry; the other five providers' results are returned unchanged (Table VII)	Semantic Scholar returned HTTP 429; results from the other four providers (8 papers) were still delivered	Qualitatively consistent. Whether the retry fired, and the elapsed time, were not recorded, so timing is not compared
Per-provider result cap	The orchestrator keeps at most k papers per provider before deduplication	Each of four providers returned 2 papers	Consistent with a per-provider cap if the limit was 2 (the limit was not recorded); the effect of deduplication on the 8 results is unknown
AI pipeline scheduling	Serial loop; start-to-start spacing $\max(L, \tau)$ with τ=2 s; total $(N-1)\max(L, \tau) + L$ (Figure 6a)	8 papers analyzed and saved in 27 s, i.e., ≈3.4 s per paper	Consistent with a serial loop in which the per-paper cost (≈3.4 s, including saving) exceeds τ, so that τ is not binding and $T \approx N \cdot 3.4$ s (inference; per-paper latencies were not recorded)
Persistence	Insert/select path correct and fast against a local PostgreSQL (Table X)	PostgreSQL initialized; retrieved papers appeared in the Library	Functional confirmation on the real application; no timing was recorded
PDF acquisition	Not exercised in the cloud; repository tests use local fixtures	PDF retrieval worked for OpenAlex and arXiv records	Functional evidence only; counts, sizes and other providers were not recorded
Search latency	Parallel model with a 350 ms PubMed floor and arXiv throttle (Figure 3)	Not recorded	No comparison possible
Editor start-up	Not evaluated in the cloud	Approximately 1–2 s	Approximate user observation; no emulated counterpart

run, the pipeline from four live providers through PostgreSQL and the Library to Groq-based report generation worked end to end, that a rate-limited provider did not prevent the others from delivering, and that the observed AI time per paper is compatible with a serial loop. The code's own error message for Semantic Scholar recommends configuring an API key for a dedicated allowance (`semanticscholar.js`, class [C]); the live HTTP 429 is consistent with that advice but does not establish the cause. The session does *not* establish typical latency, success rates, duplicate rates, relevance of the retrieved papers, or the accuracy of any generated analysis, and one query cannot support generalization.

VII. DISCUSSION

Answers to the research questions. *RQ1:* search latency is the maximum, not the sum, of adapter latencies, but the maximum includes adapter-internal serialization (PubMed spacing, Europe PMC enrichment, arXiv throttle) and has no deadline, so one hung provider holds back all results until its timeout (Section VI); failure containment is good for five providers and misreported for arXiv. *RQ2:* the deduplicator reliably merges records that share a DOI and nothing else; for DOI-less records it merges only metadata-identical strings, and the outcome depends on input order. *RQ3:* local compute is negligible next to network time, but the library view re-reads and scans the whole table and scales with rows × files in its fallback path. *RQ4:* the AI client enforces a τ-spaced serial schedule and bounded back-off for 429 only; it sends the literal string `null` for a missing abstract. The single live session (Section VI-F) did not contradict any of these emulated findings: a rate-limited provider did not block the others, and the observed 27 s for eight analyses is compatible with a serial loop.

Emulated versus live evidence. The emulated experiments establish *mechanisms* (what the code does when a provider is slow, fails, or disagrees on metadata) under controlled conditions, with exact counts that an independent re-run reproduced. The live session adds what they cannot: evidence that the full pipeline works against real providers, a real database initialization, real PDF hosts and the real Groq endpoint on a Windows desktop. It cannot establish rates, distributions or generality, and the two kinds of evidence answer different questions, so we compare them only qualitatively (Table XII). Two quantities the live session would need to record to allow quantitative comparison are the per-provider latency of a search and the per-paper latency of the AI call; the application records neither (`providerStats` holds only status and counts), so a future live protocol would need timestamps added around the adapter calls and the AI request.

Engineering recommendations tied to evidence. (1) Make adapter failure handling uniform: the arXiv adapter should reject on HTTP errors and timeouts like the other five. (2) Add a global deadline with progressive delivery, since a single hung provider costs 15–33 s, whereas healthy emulated providers settle in well under a second (real settle times were not measured); because hedged requests [19] would multiply load on rate-limited free APIs, a deadline plus partial results is the proportionate response. (2b) Configure a Semantic Scholar API key, as the code's own error message advises, since an unauthenticated provider was rate limited in the live session. (3) Replace first-match deduplication by transitive closure over registered keys of *all* records, which makes the output order independent, costs the same $O(n)$ (Figure 5), and lets a DOI veto and empty-field guards be added. (4) Any relaxation of the fallback key (diacritic folding, punctuation-insensitive titles, surname/initial authors, ± 1 year) recovers the DOI-less

cases but admits same-initial/same-title false merges, so it should be adopted only with a labelled evaluation on real provider data and extra evidence such as author-list overlap. (5) Bound or defer the Europe PMC enrichment fan-out (k extra Crossref requests per search). (6) Guard null abstracts and over-long inputs in the prompt, honour `Retry-After`, and retry transient 5xx responses. (7) Batch database inserts and index the library scan if libraries grow beyond a few thousand records. (8) Call arXiv over HTTPS: `arxiv.js` uses a plain-HTTP URL, which an HTTPS-only egress proxy could not carry in our test environment [C]. These are suggestions derived from measured behaviour [S], the live session [L] and code inspection [C]; we did not implement them in the application, and we make no claim that they improve it, since no comparative evaluation was run.

Engineering contribution vs. scientific novelty. The system itself is an integration of existing services and techniques; its value is practical. The scientific content of this paper is limited to (i) an explicit, tested latency model of a federated fan-out with adapter-internal throttling, (ii) a demonstration that a common "first match wins" key-based deduplicator is non-confluent and a quantification of which metadata disagreements defeat equality keys, and (iii) a reusable fault-injection harness. None of these is a new algorithm, and none establishes retrieval or summarization quality.

VIII. Limitations and Threats to Validity

Emulated upstreams and one live session. Upstream behaviour in all [S] experiments is emulated. Provider latency, rate-limit behaviour, error rates, payload quirks and genuine cross-provider overlap are therefore unmeasured there; injected latencies are parameters, not estimates of real services, and agreement with Equations (1) and (2) shows that the code behaves as modelled, not that real searches take any particular time. The only live evidence [L] is one user-recorded session (one query, no repetition) taken from in-app logs and not independently reproduced; it did not record search latency, per-paper AI latency, the configured limit, PubMed participation, PDF counts, or system details (Section VI-F), the AI timing covers processing and saving and not accuracy, and the editor timing is approximate. The LLM endpoint was likewise only emulated in the cloud: we measured scheduling and request construction, not output quality.

Reconstructed payloads. Raw provider responses are rendered from the fields that the repository's normalizers read and from its own test fixtures. If real responses differ (e.g., author-name conventions, title punctuation, year semantics), residual-duplicate rates will differ. Single-factor perturbations (Figure 4) are deterministic properties of the key definitions and do not depend on assumed rates, but the mixed scenario of Table IX depends on the assumed preprint-year skew (40 % of arXiv-bearing works) and abbreviated Europe PMC author strings, and must be read as illustrative.

Synthetic corpus and no relevance judgments. Titles are unique by construction, which removes natural-language near-duplicates (e.g., translated titles, errata, "Part I/II" series), so false-merge rates for the strict key are lower bounds on what real corpora would show and those for the relaxed key are probes, not estimates. We evaluate no query relevance, no recall against a gold standard, and no ranking quality; the year-based re-ranking may discard relevant older papers, which we neither confirm nor refute.

Prototype variants. UF-REPO and UF-RELAXED exist only in the harness; UF-RELAXED was designed after inspecting the failure cases and tuned to nothing else, but it is untested on real data and deliberately lacks safeguards (author-list overlap, DOI veto) that a production resolver would need.

Environment. One machine (4 vCPU), one Node.js version, one PostgreSQL version on loopback; absolute microsecond-level timings will differ elsewhere, and PostgreSQL latency over a network would change Table X. The Electron GUI, the editor, Pandoc export, and the PDF download path against real publishers were not exercised; the security posture (plaintext key storage, broad IPC surface) was read from the code, not tested. Code inspection also shows that the PDF download `fetch` has no timeout or size limit; we did not test this.

IX. Conclusion

We characterized the Research Intelligence Engine by reading its code and by running its unmodified modules against an emulated upstream environment and a real PostgreSQL server. Search latency follows the maximum of adapter latencies to within about 11 ms in our sweep, but provider-internal throttles impose a 350 ms floor and up to ≈2.6 s of extra delay for back-to-back searches, a hung provider delays the whole result set until its timeout, and the arXiv adapter reports failures as empty successes. The deduplicator is order dependent and recovers no DOI-less duplicate under any single-factor metadata disagreement except letter case; a union-find resolver over the same keys is order independent at equal cost, and a relaxed-key prototype recovers most DOI-less cases at the price of measurable false merges. Local compute and persistence costs are small. The AI client schedules serially with a 2 s interval and bounded exponential back-off, but silently sends `null` for missing abstracts. A single user-recorded live session on a Windows desktop (four providers returning two papers each, Semantic Scholar rate limited, eight papers analyzed and saved in 27 s) was consistent with these findings without being a benchmark. What remains open is everything that requires repeated live measurement: real latency distributions, real overlap and duplicate rates, retrieval effectiveness, and the quality of LLM analyses. The accompanying harness is intended to make such measurements easy to add once network access is available.

Reproducibility and Availability

All files are in the `paper_exp/` directory of the repository: `main.tex`, `references.bib`, `figures/` (SVG sources, generators, PDF conversions), `experiments/` (harness; `run_all.sh` reproduces every [S] result in about 25 minutes, mostly deliberate waits), `results/` (raw JSON for every table and figure, plus `environment.json`), `logs/`,

and `verification/` (claim-by-claim audit, module-usage report, an independent re-run, and the bibliographic audit). No application source file was modified (checked against baseline commit `bf2d3a3`). Requirements: Node.js $\geq$18, `npm ci --omit=dev --ignore-scripts`, and a local PostgreSQL with database `rie_bench` for E4. The AI experiments use a dummy key and an emulated endpoint; no credential was used. The repository tests [R] need network access and, in the cloud sandbox, the proxy variable and shim described in Section V (`logs_online/`); the live observations [L] cannot be reproduced in the cloud; they are recorded in Table XI as reported by the author from the in-app logs.

REFERENCES

[1] J. Priem, H. Piwowar, and R. Orr, "OpenAlex: A fully-open index of scholarly works, authors, venues, institutions, and concepts," 2022, arXiv:2205.01833. doi: https://doi.org/10.48550/arXiv.2205.01833.

[2] K. Lo, L. L. Wang, M. Neumann, R. Kinney, and D. Weld, "S2ORC: The Semantic Scholar open research corpus," in *Proc. 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 4969–4983, doi: https://doi.org/10.18653/v1/2020.acl-main.447.

[3] R. Kinney *et al.*, "The Semantic Scholar open data platform," 2023, arXiv:2301.10140. doi: https://doi.org/10.48550/arXiv.2301.10140.

[4] G. Hendricks, D. Tkaczyk, J. Lin, and P. Feeney, "Crossref: The sustainable source of community-owned scholarly metadata," *Quantitative Science Studies*, vol. 1, no. 1, pp. 414–427, 2020, doi: https://doi.org/10.1162/qss_a_00022.

[5] C. Ferguson *et al.*, "Europe PMC in 2020," *Nucleic Acids Research*, vol. 49, no. D1, pp. D1507–D1514, 2021, doi: https://doi.org/10.1093/nar/gkaa994.

[6] E. Sayers, "A general introduction to the E-utilities," Entrez Programming Utilities Help, National Center for Biotechnology Information (NCBI Bookshelf NBK25497), 2022, created 2009, updated 17 Nov. 2022. Available: https://www.ncbi.nlm.nih.gov/books/NBK25497/.

[7] arXiv, "Terms of use for arXiv APIs," https://info.arxiv.org/help/api/tou.html, legacy APIs (including the arXiv API): no more than one request every three seconds, single connection. Accessed Nov. 2026.

[8] M. Visser, N. J. van Eck, and L. Waltman, "Large-scale comparison of bibliographic data sources: Scopus, Web of Science, Dimensions, Crossref, and Microsoft Academic," *Quantitative Science Studies*, vol. 2, no. 1, pp. 20–41, 2021, doi: https://doi.org/10.1162/qss_a_00112.

[9] M. Gusenbauer and N. R. Haddaway, "Which academic search systems are suitable for systematic reviews or meta-analyses? Evaluating retrieval qualities of Google Scholar, PubMed, and 26 other resources," *Research Synthesis Methods*, vol. 11, no. 2, pp. 181–217, 2020, doi: https://doi.org/10.1002/jrsm.1378.

[10] J. P. Alperin, J. Portenoy, K. Demes, V. Larivière, and S. Haustein, "An analysis of the suitability of OpenAlex for bibliometric analyses," 2024, preprint. arXiv:2404.17663. doi: https://doi.org/10.48550/arXiv.2404.17663.

[11] Z. Ji *et al.*, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 248:1–248:38, 2023, doi: https://doi.org/10.1145/3571730.

[12] X. Tang, X. Duan, and Z. Cai, "Large language models for automated literature review: An evaluation of reference generation, abstract writing, and review composition," in *Proc. 2025 Conf. on Empirical Methods in Natural Language Processing (EMNLP)*. Suzhou, China: Association for Computational Linguistics, 2025, pp. 1602–1617, doi: https://doi.org/10.18653/v1/2025.emnlp-main.83.

[13] I. P. Fellegi and A. B. Sunter, "A theory for record linkage," *Journal of the American Statistical Association*, vol. 64, no. 328, pp. 1183–1210, 1969, doi: https://doi.org/10.2307/2286061.

[14] A. K. Elmagarmid, P. G. Ipeirotis, and V. S. Verykios, "Duplicate record detection: A survey," *IEEE Transactions on Knowledge and Data Engineering*, vol. 19, no. 1, pp. 1–16, 2007, doi: https://doi.org/10.1109/TKDE.2007.250581.

[15] P. Christen, *Data Matching: Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection*. Berlin, Heidelberg: Springer, 2012, doi: https://doi.org/10.1007/978-3-642-31164-2.

[16] H. Piwowar, J. Priem, V. Larivière, J. P. Alperin, L. Matthias, B. Norlander, A. Farley, J. West, and S. Haustein, "The state of OA: a large-scale analysis of the prevalence and impact of Open Access articles," *PeerJ*, vol. 6, p. e4375, 2018, doi: https://doi.org/10.7717/peerj.4375.

[17] M. Shokouhi and L. Si, "Federated search," *Foundations and Trends in Information Retrieval*, vol. 5, no. 1, pp. 1–102, 2011, doi: https://doi.org/10.1561/1500000010.

[18] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. 32nd Int. ACM SIGIR Conf. on Research and Development in Information Retrieval*, 2009, pp. 758–759, doi: https://doi.org/10.1145/1571941.1572114.

[19] J. Dean and L. A. Barroso, "The tail at scale," *Communications of the ACM*, vol. 56, no. 2, pp. 74–80, 2013, doi: https://doi.org/10.1145/2408776.2408794.

[20] W. M. Bramer, D. Giustini, G. B. de Jonge, L. Holland, and T. Bekhuis, "De-duplication of database search results for systematic reviews in EndNote," *Journal of the Medical Library Association*, vol. 104, no. 3, pp. 240–243, 2016, doi: https://doi.org/10.3163/1536-5050.104.3.014.

[21] J. Rathbone, M. Carter, T. Hoffmann, and P. Glasziou, "Better duplicate detection for systematic reviewers: evaluation of Systematic Review Assistant-Deduplication Module," *Systematic Reviews*, vol. 4, p. 6, 2015, doi: https://doi.org/10.1186/2046-4053-4-6.

[22] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020, arXiv:2005.11401. https://papers.nips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html.

[23] Y. Gao *et al.*, "Retrieval-augmented generation for large language models: A survey," 2023, arXiv:2312.10997.

[24] J. Lála, O. O'Donoghue, A. Shtedritski, S. Cox, S. G. Rodriques, and A. D. White, "PaperQA: Retrieval-augmented generative agent for scientific research," 2023, arXiv:2312.07559.

[25] M. D. Skarlinski *et al.*, "Language agents achieve superhuman synthesis of scientific knowledge," 2024, arXiv:2409.13740.

[26] J. M. Nicholson, M. Mordaunt, P. Lopez, A. Uppala, D. Rosati, N. P. Rodrigues, P. Grabitz, and S. C. Rife, "scite: A smart citation index that displays the context of citations and classifies their intent using deep learning," *Quantitative Science Studies*, vol. 2, no. 3, pp. 882–898, 2021, doi: https://doi.org/10.1162/qss_a_00146.

[27] OpenAI, "gpt-oss-120b & gpt-oss-20b model card," 2025, arXiv:2508.10925.